

Lernziele, hier erfahren Sie,

- wie Software funktioniert und wie diese mit *maschinennahen* oder *höheren* Programmiersprachen entwickelt werden kann.
- welche wesentlichen Bestandteile Software identifiziert und deren Interaktionen.

Ausgangssituation:

Was hat das mit Softwareentwicklung zu tun?

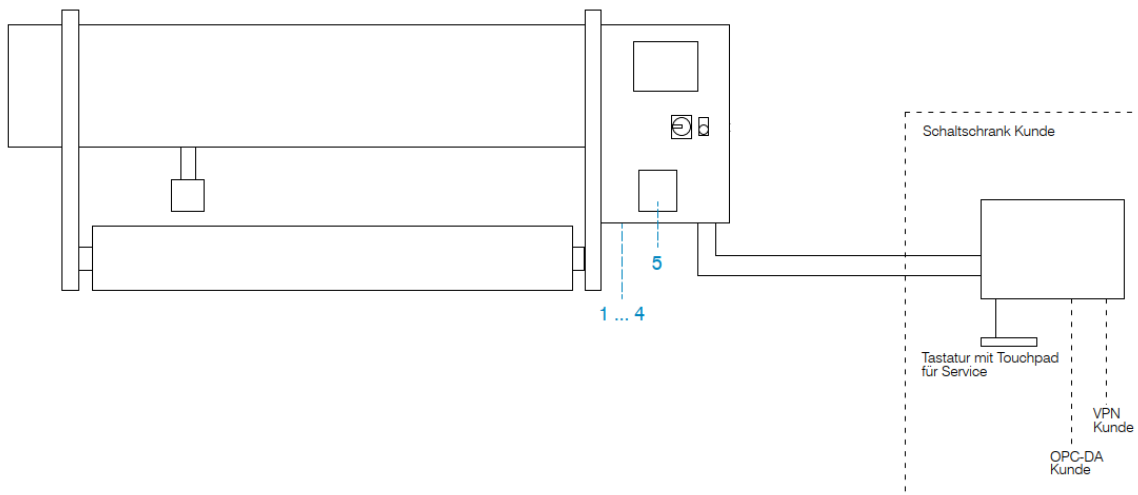
Warum ist Softwareentwicklung schwierig?

Wie können diese Probleme beherrschbar gemacht werden?

Welche Ziele und Mittel verfolgt somit das Software Engineering?

- - Weil Softwareentwicklung viele Anforderungen, komplexe Systeme und wechselnde Rahmenbedingungen berücksichtigen muss - Fehler oder Missverständnisse wirken sich stark auf das Ergebnis aus.
- - Durch systematisches Vorgehen: klare Anforderungen, sauberes Design, strukturierte Programmierung, Tests und Dokumentation.
- - Ziele: Qualität, Zuverlässigkeit, Wartbarkeit und Effizienz der Software.

Stellen Sie sich vor, Sie sind als späterer Fachinformatiker in einem Messtechnik-Unternehmen beschäftigt, das Anlagen zur **Dickenmessung** herstellt. Im Zuge des mechanischen und elektrotechnischen Aufbaus müssen auch die **Schnittstellen** zwischen den einzelnen Teilsystemen (Mechanik, Schaltschrank => Elektrik und PC/Computer) berücksichtigt werden. Ihre **Aufgabe** ist es nun die Anlage PC-gerecht auszustatten, die richtigen **PC-Komponenten** zur Verfügung zu stellen und auszuwählen.



Dabei stellt das **HMI** immer die Bedienerschnittstelle zur Anlage dar. Im Bild oben sehen Sie, wie so eine **Schnittstellenbeschreibung** nun aussehen könnte.

Der PC ist entsprechend mit dem HMI am Schaltschrank der Anlage verbunden. Am PC wiederum werden Maus, Tastatur etc. angeschlossen, um die Anlage über die **Software bedienen** zu können und die **Dicke des Materials** über verschiedene Auswerteverfahren messen zu können.

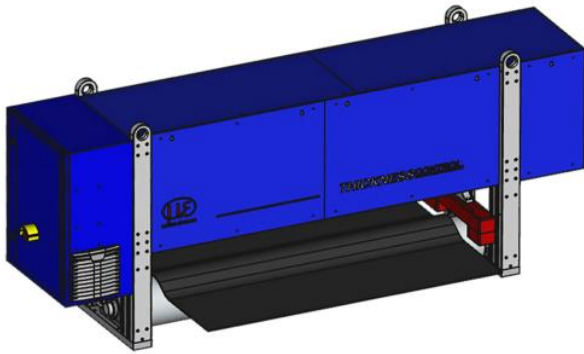


Machen Sie sich Notizen!

Software Engineering: Bestandteile von Software

Aufbau und Funktionsweise von Software

1	Not-Halt Kunde	4	Erdung
2	Digital Ein/Aus Kunde	5	Druckluftanschluss
3	230V 10A Kunde	---	nicht im Lieferumfang enthalten



Was ist Software? Um welche Art von Software handelt es sich hier?

Was ist Software?

Welche Systematik von Software kommt hier zum Einsatz?

Welche Vor- und Nachteile haben unterschiedliche Softwarearten?



Machen Sie sich Notizen!



- - Was ist Software?

In der Ausgangssituation sind es Programme, die die Dickenmessanlage steuern, Messwerte erfassen und für den Bediener am PC/HMI darstellen.

- - Welche Systematik von Software kommt hier zum Einsatz?

- **Systemsoftware:** Betriebssystem und Treiber, die den PC mit der Anlage (HMI, Schaltschrank) verbinden.
- **Anwendungssoftware:** spezielle Mess- und Auswertesoftware zur Bedienung und Analyse der Dickenmessung.

- - Vor- und Nachteile unterschiedlicher Softwarearten in diesem Kontext:

- **Standardsoftware** (z. B. Windows, Office, ggf. Standarddatenbanken): schnell verfügbar, kostengünstig, stabil – aber nicht speziell auf die Anlage zugeschnitten.
- **Individualsoftware** (Mess- und Bedien-Software): exakt auf die Anforderungen der Anlage abgestimmt, ermöglicht präzise Steuerung und spezielle Auswerteverfahren – jedoch aufwändig in Entwicklung, Pflege und teurer.

Aufbau und Funktionsweise von Software

Welche Hardware umfasst die Ihnen vorliegende Industrieanlage?

Welche Software könnte die vorstehende Industrieanlage beinhalten?

Hardware

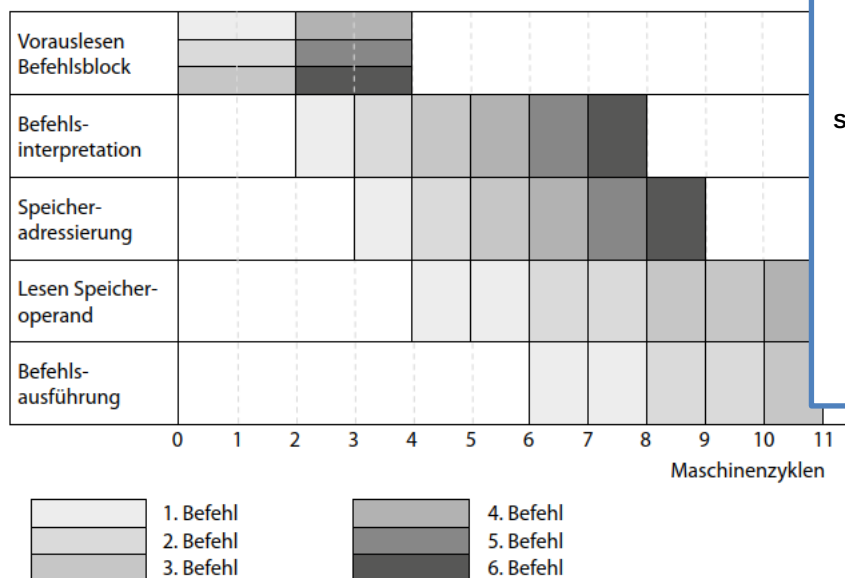
= alle physisch, materiell angreifbaren Komponenten

Software

= alle immateriellen Anteile, welche die Funktionsweise der Industrieanlage steuern

Was ist Software und wie entsteht Software?

Dazu muss man wissen, wie ein **Prozessor** funktioniert.



Hardware der Industrieanlage (Ausgangssituation):

- **Mechanik:** Anlage zur Dickenmessung (Sensoren, Messvorrichtung).
- **Schaltschrank / Elektrik:** Steuerung, HMI (Bedienfeld), Netzteile, SPS.
- **PC/Computer:** mit Maus, Tastatur, Monitor.
- **Schnittstellen:** Kabel/Adapter, die PC, Schaltschrank und HMI verbinden.

Software der Industrieanlage:

- **Systemsoftware:** Betriebssystem des PCs (z. B. Windows), Treiber für Schnittstellen/HMI.
- **Anwendungssoftware:** spezielle Mess- und Steuerungssoftware, Auswerteverfahren (z. B. zur Darstellung und Analyse der Dickenwerte).
- **Firmware:** in SPS, Sensoren oder HMI, die Grundfunktionen steuert.

Wie Sie erkennen können, ist es möglich, dass von einem Prozessor mehrere Maschinenbefehle gleichzeitig verarbeitet werden.

In unserem Beispiel können maximal **drei Befehle zeitgleich** aus dem Zentralspeicher gelesen werden, wobei der erste Befehl nach acht Maschinenzyklen abgeschlossen ist, der zweite nach zehn Zyklen.

Ohne Fließbandverarbeitung hätte die Abarbeitung des zweiten Befehls erst nach dem Abschluss des ersten beginnen können, und er könnte erst zum Zeitpunkt 18 abgeschlossen werden. Nach dem vierten Maschinenzyklus könnten bereits weitere Befehlsverarbeitungen beginnen.

Was passiert hier?

1. Ein auf einem Rechner ausführbares Programm besteht aus einer Reihe von detaillierten Anweisungen an den Rechner in Form von **Maschinenbefehlen**.
2. Damit ein Programm (kann nicht nur *Anweisungen*, sondern auch *Daten* oder *Speicheradressen* beinhalten) ausgeführt werden kann, muss es zuerst in den Arbeitsspeicher geladen werden.
3. Das Maschinenprogramm ist letztendlich eine Folge von **Nullen und Einsen**, die vom Prozessor interpretiert werden.

4. Wenn Menschen Programme als Folge von *binären* Zeichen erstellen würden, wäre dies sehr **aufwendig und fehleranfällig**. Programme müssen von Menschen gelesen und verstanden werden, um diese weiterzuentwickeln, um *Fehler* zu erkennen und *Veränderungen* in den Programmen durchführen zu können.

Warum glauben Sie, sollte Software wartbar und anpassbar sein?

Was könnte man bei der vorstehenden Industrieanlage unter einer kundenspezifischen Softwareanpassung verstehen?

Welchen Mehrwert hat das für die Softwareentwicklung?

-- Warum sollte Software wartbar und anpassbar sein?

Weil sich Anforderungen, Technik und Kundenwünsche ändern – die Software muss leicht erweitert, korrigiert oder aktualisiert werden können.

-- Kundenspezifische Softwareanpassung bei der Industrieanlage:

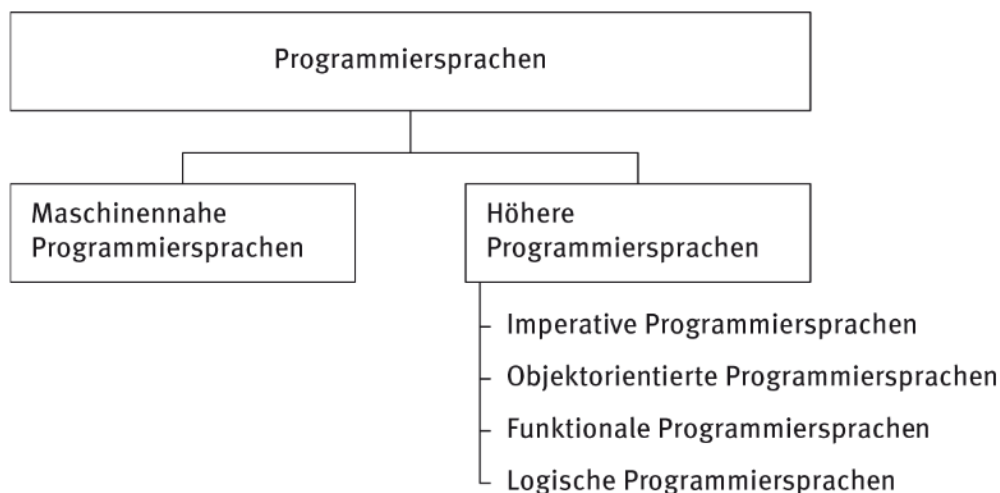
Beispiel: Erweiterung der Mess-Software um zusätzliche Auswerteverfahren, Integration neuer Sensortypen, Anpassung der HMI-Oberfläche an die Bedürfnisse des Bedieners oder Anbindung an ein ERP-System des Kunden.

-- Mehrwert für die Softwareentwicklung:

- Höhere Kundenzufriedenheit durch passgenaue Lösungen
- Längere Nutzungsdauer der Software (Investitionsschutz)
- Flexibilität bei zukünftigen Änderungen
- Wettbewerbsvorteil durch maßgeschneiderte Funktionen

Programmiersprache

Eine Programmiersprache ist deshalb eine Sprache zur Formulierung von Rechenvorschriften (Algorithmen), die von einem Computer ausgeführt werden können. Mittels Programmiersprachen werden komplexe Aufgabenstellungen durch eine detaillierte Folge von Rechanweisungen und Datenbeschreibungen in einer automatisierbaren Weise definiert.



Höhere Programmiersprachen sind weitgehend unabhängig von dem zugrundeliegenden **Prozessortyp**. Während ein Quellprogramm in maschinennaher Sprache sich auf Eigenschaften des Prozessors bezieht (beispielsweise Befehlssatz und Register), fehlen diese Elemente in höheren Programmiersprachen völlig.

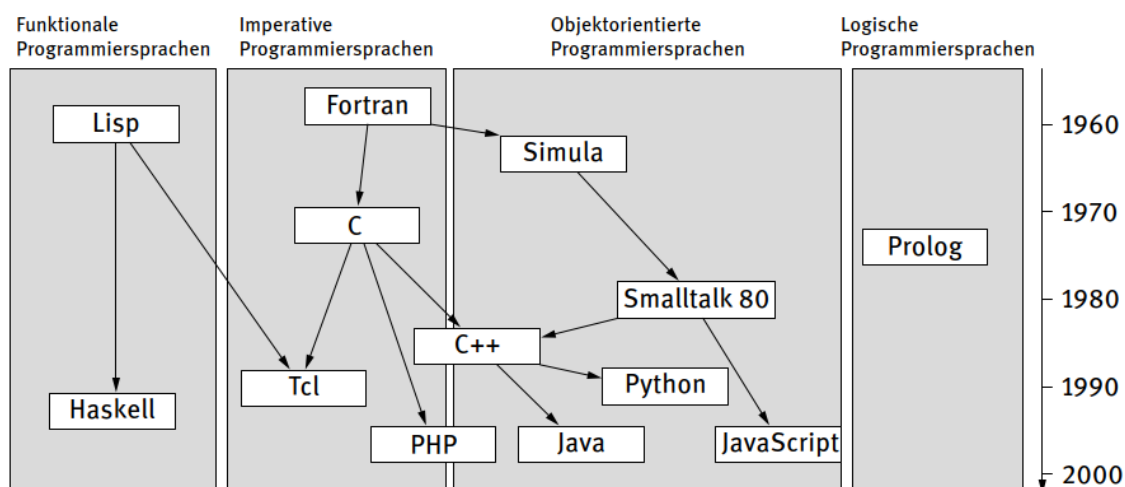
Stattdessen werden dort **Variable und Konstante** in unterschiedlichen Datentypen definiert und für die **Ablaufsteuerung** und Berechnungen werden prozessorunabhängige Befehle verwendet.

Wie könnten sich die o. g. höheren Programmiersprachen nun unterscheiden?

Machen Sie eine Kurz-Recherche zu folgenden Begriffen: imperative Programmiersprachen, objektorientierte Programmiersprachen, funktionale Programmiersprachen, logische Programmiersprachen

Lehrer: Zusammentragen der Ergebnisse

- **Imperative** Programmiersprachen: Im Vordergrund steht die Definition von komplexen Anweisungen an den Rechner, wie beispielsweise Kopieren von Speicherinhalten, Lesen von Dateien oder Versenden von Datenblöcken über ein Netzwerk.
- **Objektorientierte** Programmiersprachen: Im Vordergrund steht eine Sichtweise auf komplexe Programmsysteme auf der Basis von kommunizierenden Objekten, die teilweise den Objekten der abgebildeten Realwelt entsprechen. Objekte mit gleichen Eigenschaften werden zu Klassen zusammengefasst, die Methoden (aufrufbare Funktionen) bereitstellen.
- **Funktionale** Programmiersprachen: Im Vordergrund steht die Definition von komplexen Funktionen, wie beispielsweise: Multipliziere bei der Ermittlung des Mietpreises das Ergebnis der Flächenberechnung mit dem Quadratmeterpreis.
- **Logische** Programmiersprachen: Im Vordergrund steht die Definition von logischen Zusammenhängen des modellierten Realweltausschnitts, wie beispielsweise, dass jede männliche Person, die ein Kind hat, als Vater bezeichnet wird.



Wie wird nun ein Quellprogramm aus einer höheren Programmiersprache übersetzt?

- Eine Sprache, deren Befehle sich am Anwendungsbereich orientieren, wird als **höhere Programmiersprache** (engl.: higher level programming language) bezeichnet. Höhere Programmiersprachen abstrahieren vom Befehlssatz des Prozessors und verwenden Elemente der natürlichen Sprache und/oder der Mathematik für das Formulieren von Programmen.
- Ein **Compiler** (auch: Kompilierer, engl.: compiler) ist ein Übersetzungsprogramm, das ein in einer höheren Programmiersprache abgefasstes Quellprogramm in eine Maschinensprache übersetzt (kompiliert). Die Übersetzung erfolgt häufig über den Zwischenschritt der Übersetzung in eine maschinennahe Sprache, wobei dann ein Assembler zur Erzeugung des Maschinencodes herangezogen wird.
- Ein **Binder** (engl.: linker) ist ein Dienstprogramm, das mehrere kompilierte Teilprogramme zu einem gemeinsamen Binärprogramm zusammenfasst und dabei Verweise (wie beispielsweise Funktionsnamen) auf externe Softwarekomponenten durch Sprungadressen ersetzt. Gleichzeitig werden auch Speicheradressen von Variablen ermittelt und im Programm eingesetzt.
- Ein **Lader** (engl.: loader) ist ein Dienstprogramm, das ein Binärprogramm von einem externen Speichermedium liest und in den Arbeitsspeicher kopiert, Kommandozeilenargumente übergibt und Register initialisiert, und schließlich das Programm startet (an die Startadresse des Programms verzweigt).
- Ein **Interpreter** (engl.: interpreter) ist ein Programm, das ein in einer höheren Programmiersprache abgefasstes Quellprogramm bei der Ausführung laufend interpretiert, das heißt, die Anweisungen in dem Programm laufend überprüft, analysiert und entsprechende Operationen vornimmt. Meist wird dabei aus Effizienzgründen beim Laden das Quellprogramm in einen *Zwischencode* (engl.: intermediate code) übersetzt, der interpretiert wird, wodurch wiederholte Überprüfungen von Anweisungen reduziert werden können.

Was könnten in diesem Kontext nun Softwarekomponenten sein?

Lesen Sie sich obige Folie gut durch und erörtern Sie für sich die Frage, was in diesem Kontext Softwarekomponenten sein könnten.

--Allgemein: Was sind Softwarekomponenten?

- Definition: Abgrenzbare, wiederverwendbare Teile einer Software, die bestimmte Funktionen bereitstellen.
- Sie haben klare Schnittstellen (Input/Output), damit sie im Gesamtsystem zusammenspielen.
- Beispiel allgemein: Datenbankmodul, Kommunikationsschnittstelle, Benutzeroberfläche.

Welche Softwarekomponenten könnten bei unserer Industrieanlage zum Einsatz kommen?

--Treiberkomponenten

- z. B. Treiber für die Verbindung zwischen PC und HMI/Schaltschrank.

--Kommunikationsmodule

- Schnittstelle zur SPS oder zu Mess-Sensoren (z. B. Profinet-, Modbus- oder Ethernet-Komponente).

--HMI-Komponente

- Visualisierung und Bedienoberfläche (Anzeige von Messwerten, Eingabe von Parametern).

--Auswertemodule

- Softwareteile zur Berechnung und Analyse der Dickenwerte (z. B. Statistik, Mittelwert, Toleranzprüfung).

--Datenmanagement

- Komponente zur Speicherung, Protokollierung und Archivierung von Messergebnissen (lokal oder in einer Datenbank).

--Schnittstellen zu Fremdsystemen

- z. B. Export von Messdaten an ein ERP-System oder Qualitätsmanagementsystem.

Softwarekomponenten

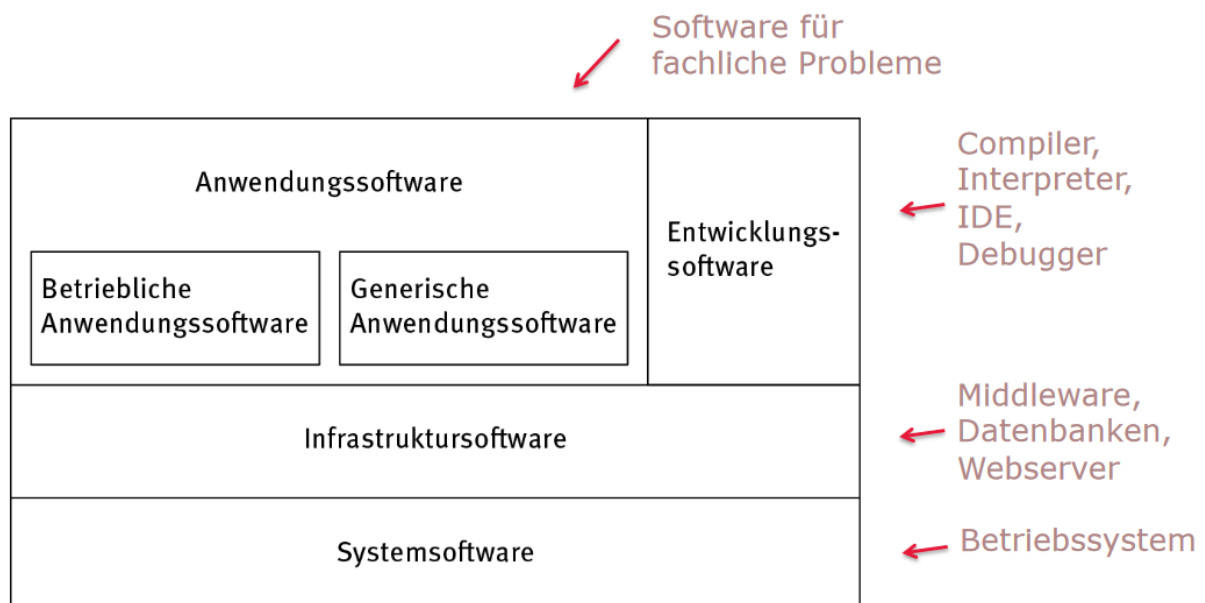
Def.: In komplexen Programmsystemen treten immer wiederkehrende Aufgabenstellungen auf, die man aus Gründen der Fehler- und Aufwandsreduktion am besten an einer Stelle möglichst allgemein löst, auch weil man später diese Lösung möglichst oft **wiederverwenden** möchte.

Quellprogramme können sich über Namen auch auf Funktionen beziehen, die nicht in dem Quellprogramm definiert sind.

Auf diese Weise kann ein Programm auf andere **Softwarekomponenten** (beispielsweise *Programmbibliotheken* für Ein-/Ausgabe oder Arithmetik) zurückgreifen und diese wiederverwenden.

Bestandteile von Software

Software ist meist kein untrennbares Ganzes, sondern besteht aus einer Vielzahl von interagierenden Softwarekomponenten.



Welche Software könnte bei unserer vorstehenden Industrieanlage zum Einsatz gekommen sein?

Lehrer: Zusammentragen der Ergebnisse

- Die **Systemsoftware** stellt die grundlegenden Funktionen für den Betrieb eines Rechners zur Verfügung. Wichtige Bestandteile sind das Betriebssystem (engl.: operating system) und Komponenten für die Ansteuerung von Geräten, falls diese nicht direkt vom Betriebssystem unterstützt werden.

Das Betriebssystem ist auf die jeweilige Rechnerplattform abgestimmt und zum Betrieb eines Rechners unbedingt nötig. Das Betriebssystem bildet eine Abstraktionsschicht von der Rechnerhardware, sodass die gleichen Programme auch auf unterschiedlicher Hardware lauffähig sind. Zudem steuert und überwacht das Betriebssystem die Ausführung von Anwendungsprogrammen.

- Die **Infrastruktursoftware** (Synonym: Middleware; engl.: middleware) stellt die technische Infrastruktur für andere Komponenten zur Verfügung. Diese technische Infrastruktur bietet ohne Anwendungskomponenten für

Software Engineering: Bestandteile von Software

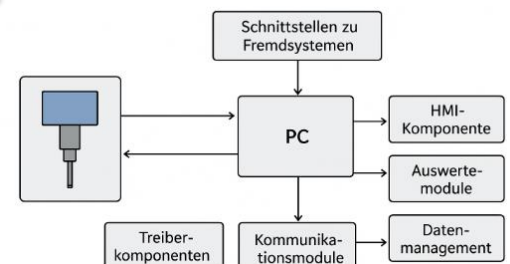
Aufbau und Funktionsweise von Software

Anwender nur einen geringen Mehrwert, vereinfacht allerdings die Entwicklung und den Betrieb von Anwendungssoftware. Zu den Infrastrukturkomponenten zählen zum Beispiel Datenbankverwaltungssysteme, Web- und E-Mail-Server, grafische Benutzerschnittstellen oder Komponenten zur Netzwerkkommunikation (beispielsweise für die Unterstützung von SOA). Bildlich gesprochen befindet sich die Middleware somit „in der Mitte“ zwischen den Softwarekomponenten, die sie verbindet. Die Infrastruktursoftware baut auf dem Betriebssystem auf.

- Die **Anwendungssoftware** (engl.: application software; Synonym: Anwendungsprogramm, Applikation) bietet Lösungen für fachliche Probleme. Dazu gehören technisch-wissenschaftliche Programme (zum Beispiel für statische Berechnungen), auf allgemeine betriebswirtschaftliche Funktionen bezogene Programme (zum Beispiel für die Finanzbuchhaltung) und Branchenprogramme (zum Beispiel für Lebensmittelfilialbetriebe). Während betriebliche Anwendungssoftware direkt die Funktionen und Abläufe eines Betriebs unterstützt (beispielsweise ERP-Software), bietet generische Anwendungssoftware Funktionen, die innerhalb und außerhalb eines Betriebs genutzt werden. Zu der generischen Anwendungssoftware gehören beispielsweise Textverarbeitungs- oder Tabellenkalkulationssoftware, E-Mail-Klientenprogramme oder Webbrowser.

- Die **Entwicklungssoftware** (engl.: development software) unterstützt die Programmierung prinzipiell beliebiger Programme. Ein wichtiger Teil einer Entwicklungsumgebung ist die zugehörige Programmiersprache, die aus zumindest einem Compiler/Interpreter besteht, aber auch häufig angepasste Editoren, eine Ausführungsumgebung und Fehlerbehebungssoftware (sogenannte Debugger) enthält. Mit Hilfe der Entwicklungssoftware werden sowohl System-, Anwendungs-, als auch die Entwicklungssoftware selbst erstellt.

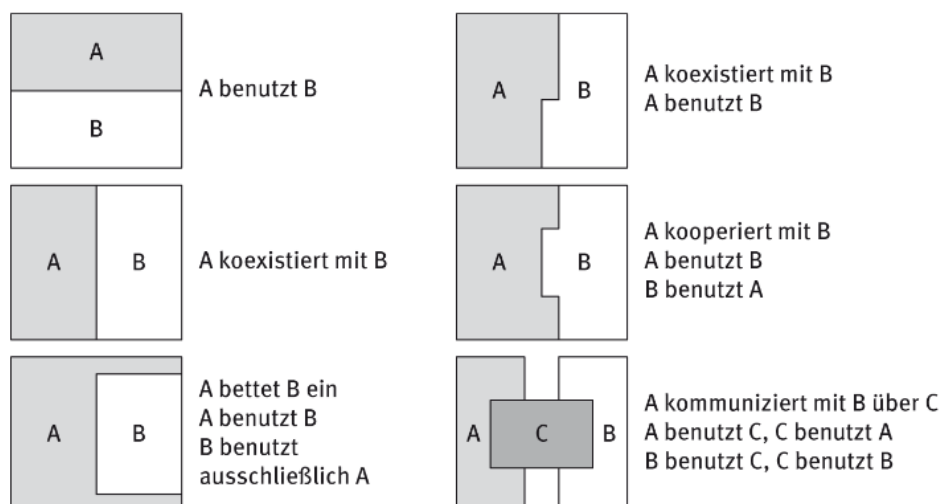
Softwarekomponenten in der Industrieanlage



Abhängigkeiten von Softwarekomponenten

Softwarekomponenten interagieren mit anderen Softwarekomponenten und bestimmen letztendlich die **Architektur** des Softwaresystems.

- Das Maß, in dem die verschiedenen Elemente innerhalb einer Softwarekomponente miteinander interagieren beziehungsweise miteinander verbunden sind, wird hierbei als **Kohäsion** (engl.: cohesion) der Komponente bezeichnet.
- Das Ausmaß, in dem eine Komponente mit anderen Komponenten interagiert, wird durch die **Kopplung** (engl.: coupling) gemessen.



Beim Architekturentwurf eines Systems wird angestrebt, die **Kohäsion** von Softwarekomponenten zu **maximieren**, während gleichzeitig die **Kopplung minimiert** werden soll. Auf diese Weise soll die Architektur des Systems möglichst leicht *verstehbar, nachvollziehbar und änderbar* gehalten werden.

Beispiel:

Wenn beispielsweise zwei Programmkomponenten sich gegenseitig Textbeschreibungen von durchzuführenden Operationen zuschicken, so ist dies eine lose Kopplung. Die Programmkomponenten kommunizieren über eine sehr einfache Schnittstelle, sie müssen sehr wenig über die Implementierung der anderen Komponente wissen. Allerdings ist das Analysieren des Textes aufwendig. Wenn die Komponenten sich gegenseitig aufrufen, so ist dies mit wesentlich weniger Rechenaufwand verbunden, die Komponenten müssen allerdings beispielsweise die Programmiersprache, die Datentypen und die Namen der Funktionen kennen. Wenn diese Programmkomponenten beispielsweise gemeinsame globale Datenstrukturen nutzen, wird die Kopplung noch höher.

Benutzungsdiagramme / Komponentendiagramme, [siehe Anhang](#)

Mit Hilfe von sogenannten Benutzungsdiagrammen können statische „benutzt“- (engl.: uses) und „besteht aus“- (engl.: consists of) Beziehungen zwischen **Komponenten** modelliert werden.

Benutzungsdiagramme bieten eine einfache aber bereits recht ausdrucksstarke Möglichkeit zur Darstellung dieser **Beziehungen**.

Aus dem Diagramm kann beispielsweise abgelesen werden, welche Komponenten voneinander unabhängig sind, und welche Komponenten von Änderungen in einzelnen Komponenten betroffen sind.

Die Notation sowie die wesentlichen Beziehungen, die in Benutzungsdiagrammen dargestellt werden können, sind o. g. Abb. visualisiert.

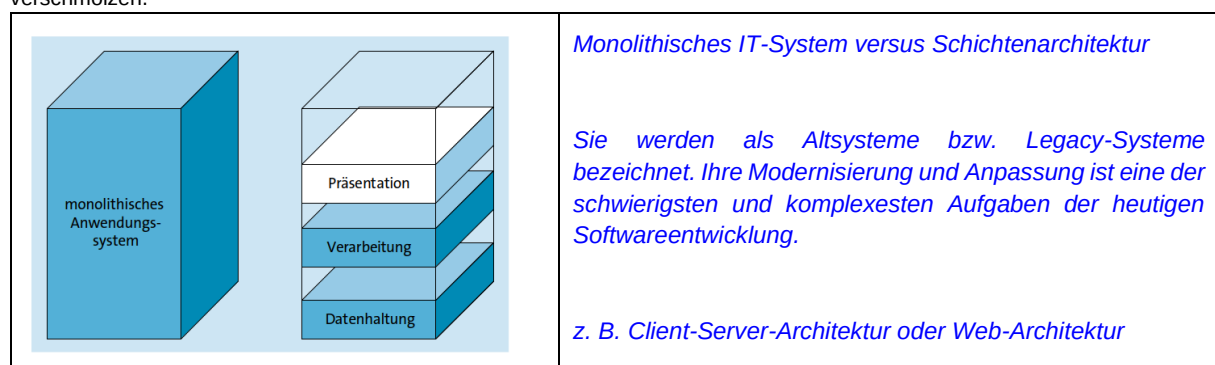
Komponenten versus monolithische (= keine Trennung) Softwareentwicklung

Salopp gesprochen folgt die komponentenorientierte Softwareentwicklung somit dem Baukastenprinzip, wobei verschiedene Bausteine prinzipiell beliebig zu neuen (größeren und mächtigeren) Bausteinen kombiniert werden können.

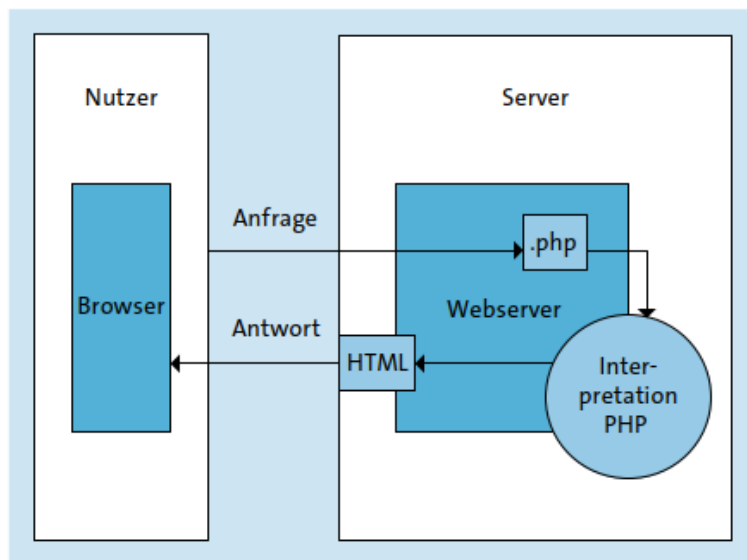
Die komponentenorientierte Softwareentwicklung hat ihren Ursprung unter anderem in der Erkenntnis, dass monolithische Systeme nur schwer wartbar, erweiterbar, testbar und wiederverwendbar sind. In der Regel müssen monolithische Systeme zu einem späteren Zeitpunkt entweder als Ganzes ausgetauscht werden, oder sie werden als Altsysteme in eine neue Umgebung eingebunden. Oft ist das letztgenannte Vorgehen notwendig, da für Altsysteme häufig keine oder eine nur sehr unvollständige Dokumentation existiert, und niemand mehr genau weiß, wie diese Systeme im Detail funktionieren.

Was sind die Nachteile von einem monolithischen IT-System?

Als monolithische IT-Systeme werden Anwendungssysteme bezeichnet, die keine explizite Trennung in unterschiedliche Teilsysteme aufweisen. Die typischen Funktionen eines Anwendungssystems, also Datenhaltung, Anwendungslogik und Benutzeroberfläche, sind hier beispielsweise eng gekoppelt und weitreichend zu einem komplexen Gesamtsystem miteinander verschmolzen.



- Das Softwaresystem ist komplex. Die fehlende Aufteilung in Einzelmodule führt dazu, dass die Komplexität nicht reduziert wird.
- Wartung und Fehlerbereinigung stellen sich sehr schwierig dar. Es gelingt nicht, einzelne Teile der Anwendung gegen neuere Technologie auszutauschen, ohne dass dies direkte Einflüsse auf andere Bestandteile der Software hat. Anpassungen an einer Stelle im Quellcode lösen eine Kette von notwendigen Folgeanpassungen aus.
- Die Wiederverwendung einzelner Softwarekomponenten ist bei einem komplizierten monolithischen Gesamtsystem ausgeschlossen. Wir können eine einmal erarbeitete Lösung nicht für eine ähnliche Problemstellung wiederverwenden, sondern müssen jedes Mal von vorn beginnen.
- Erweiterbarkeit und Skalierbarkeit sind nahezu ausgeschlossen bzw. stark eingeschränkt.



Web-Anwendungen auf der Basis von PHP

Übungen

Nehmen Sie an, ein komplexes Softwaresystem, das 300.000 Zeilen Code umfasst, soll in einem Detail erweitert werden. Welche Kriterien können oder sollen bei der Auswahl der Programmiersprache für die Erweiterungsprogrammierung herangezogen werden?

Würden Sie eher Softwarekomponenten wiederverwenden oder neu entwickeln? Welche Gründe sprechen für und gegen die Wiederverwendung von Softwarekomponenten?

Was drückt ein Benutzungsdiagramm von Softwarekomponenten aus?

Fallbezogene Aufgabe

Eine große Universität mit mehr als 20.000 Studierenden setzt für die Unterstützung der Lehr- und Lernprozesse ein umfangreiches **E-Learning-System** ein, das die Studierenden und die Lehrenden beim Lernen/Lehren und Kommunizieren über das Internet unterstützt. Das System wird auf Basis von **Open-Source-Software** gemeinsam mit anderen Universitäten weiterentwickelt und bietet lehrveranstaltungsbezogene Information (Syllabus, Kalender, An- und Abmeldung von Lehrveranstaltungen und Prüfungen, Notenbuch) und Lernmaterialien (Folien, Kontrollfragen, Musterklausuren, Lecturecasts, Aufgaben, Lern-Apps), die Unterstützung der Kommunikation in Foren oder Wikis, sowie weitere Online-Services wie Lehrveranstaltungsverzeichnis, Prüfungseinsicht, Lehrveranstaltungsevaluierung, elektronische Einreichung, Speicherung und Plagiatsüberprüfung von Abschlussarbeiten. In der Zeit der Prüfungsvorbereitung lösen Studierende bis zu 600.000 Beispiele online, wobei bis zu 2.500 Studierende gleichzeitig aktiv sind. Die durchschnittliche Antwortzeit des Systems beträgt weniger als 0,1 Sekunden.

Die Systemkonfiguration besteht derzeit aus einem **Entwicklungs- und einem Produktionsserver** mit jeweils zwei Intel-Xeon-E5-2560 Prozessoren mit je 2,6 GHz. Jeder dieser Prozessoren hat acht Kerne. Zur Erhöhung der Ausfallsicherheit dienen noch zwei weitere Rechner mit identischer Ausstattung, die im Normalbetrieb für Zusatzdienste genutzt werden. Als Speichersystem wird ein universitätsweites **SAN-Speichersystem** von IBM genutzt, das über **Glasfaserkabel** mit den **Serverrechnern** verbunden ist. Die Rechner werden unter dem Betriebssystem Linux betrieben. Das E-Learning-System speichert die Daten im **relationalen Datenbanksystem PostgreSQL**. Der Umfang der Datenbank beträgt derzeit etwa 50 GB, wobei der Umfang pro Jahr etwa 10 GB zunimmt. Die etwa 1.200 Lecturecasts liegen im Dateisystem und belegen etwa 1,5 TB. Die von den Benutzern in das System hoch geladenen Dateien (beispielsweise Studienführer, Folien, Literatur, Dateiablage, Abgabe von Hausübungen) umfassen 1,9 TB mit einem Wachstum von etwa 300 GB/Jahr. Der skalierbare, Internet-basierte Zugriff wird über einen **Webserver namens NaviServer** realisiert, der über eine **interpretierte Sprache mit Anwendungslogik** erweiterbar ist und im laufenden Betrieb weiterentwickelt werden kann. Sowohl PostgreSQL als auch der NaviServer sind in einer kompilierten Sprache implementiert, um möglichst kurze Verarbeitungszeiten zu gewährleisten. Während PostgreSQL für die Abarbeitung von **SQL-Abfragen** mehrere Prozesse verwendet, sind beim NaviServer durchschnittlich etwa 50 **Threads** gleichzeitig mit der Abarbeitung von Benutzeranfragen beschäftigt. Das E-Learning-System benutzt ein **modulares Community-System** auf Open-Source-Basis, für das mehrere hundert **Erweiterungskomponenten** entwickelt wurden. Der Gesamtumfang beträgt etwa 2 Millionen Zeilen Programmcode, von denen etwa ein Drittel im *Learn@WU-System* im Einsatz ist. Die Basiskomponenten wurden weiterverwendet, zahlreiche Komponenten wurden weiterentwickelt, einzelne Komponenten wurden neu entwickelt.

Welche Rolle spielt Modularität und Wiederverwendung von Softwarekomponenten bei dem skizzierten System? Welche Problematiken könnten in einer Softwareentwicklung entstehen, wenn nicht modular entwickelt wird?

Lösungsblatt

Zu 1.

Bei der Auswahl der Programmiersprache für die Erweiterungsprogrammierung eines komplexen Softwaresystems mit 300.000 Zeilen Code können folgende Kriterien herangezogen werden:

1. **Kompatibilität:** Die gewählte Programmiersprache sollte mit dem bestehenden Code und den verwendeten Frameworks und Bibliotheken kompatibel sein. Es sollte möglich sein, den neuen Code nahtlos in das bestehende System zu integrieren.
2. **Performance:** Je nach Anforderungen des Systems kann die Performance ein entscheidendes Kriterium sein. Manche Programmiersprachen sind schneller oder effizienter als andere. Es ist wichtig, die Auswirkungen der gewählten Programmiersprache auf die Performance des Systems zu berücksichtigen.
3. **Erfahrung und Verfügbarkeit von Entwicklern:** Es ist wichtig zu prüfen, ob es genügend Entwickler gibt, die Erfahrung mit der gewählten Programmiersprache haben. Eine Programmiersprache, für die es eine große Entwicklergemeinschaft gibt, kann von Vorteil sein, da es einfacher sein kann, Unterstützung und Hilfe zu finden.
4. **Sicherheit:** Die Sicherheit des Systems ist ein wichtiger Aspekt. Manche Programmiersprachen bieten eingebaute Sicherheitsmechanismen oder haben eine bessere Sicherheitsbilanz als andere. Es ist wichtig, die Sicherheitsaspekte der gewählten Programmiersprache zu berücksichtigen.
5. **Skalierbarkeit:** Wenn das System in Zukunft weiterwachsen und erweitert werden soll, ist es wichtig, eine Programmiersprache zu wählen, die skalierbar ist und mit den wachsenden Anforderungen des Systems umgehen kann.
6. **Wartbarkeit:** Die Wartbarkeit des Systems ist ein weiterer wichtiger Faktor. Manche Programmiersprachen sind leichter zu warten und zu debuggen als andere. Es ist wichtig, eine Programmiersprache zu wählen, die die Wartung des Systems erleichtert.
7. **Kosten:** Die Kosten für die Entwicklung und Wartung des Systems können ebenfalls ein Kriterium sein. Manche Programmiersprachen erfordern möglicherweise teure Lizenzen oder spezialisierte Entwickler, was die Kosten erhöhen kann.

Diese Kriterien sollten bei der Auswahl der Programmiersprache für die Erweiterungsprogrammierung berücksichtigt werden, um sicherzustellen, dass die gewählte Programmiersprache den Anforderungen des Systems entspricht und eine erfolgreiche Erweiterung ermöglicht.

Zu 2.

Die Entscheidung, ob Softwarekomponenten wiederverwendet oder neu entwickelt werden sollten, hängt von verschiedenen **Faktoren** ab. Es gibt sowohl Gründe, die für die Wiederverwendung von Softwarekomponenten sprechen, als auch Gründe, die dagegensprechen.

Gründe **für** die Wiederverwendung von Softwarekomponenten:

1. **Zeitersparnis:** Durch die Wiederverwendung von bereits vorhandenen Softwarekomponenten kann Zeit gespart werden, da diese bereits entwickelt, getestet und optimiert wurden. Dies ermöglicht eine schnellere Entwicklung und Markteinführung neuer Funktionen oder Produkte.
2. **Kostensparnis:** Die Wiederverwendung von Softwarekomponenten kann auch zu Kosteneinsparungen führen, da die Entwicklung neuer Komponenten vermieden wird. Dies ist besonders relevant, wenn die Entwicklung neuer Komponenten teuer oder zeitaufwändig ist.

3. **Qualitätssicherung:** Wenn Softwarekomponenten bereits erfolgreich in anderen Projekten eingesetzt wurden, kann davon ausgegangen werden, dass sie gut getestet und stabil sind. Dies kann die Qualitätssicherung des Gesamtsystems verbessern.
4. **Standardisierung:** Durch die Wiederverwendung von Softwarekomponenten können Standards und Best Practices eingehalten werden. Dies erleichtert die Wartung und Weiterentwicklung des Systems, da bereits etablierte und bewährte Lösungen verwendet werden.

Gründe **gegen** die Wiederverwendung von Softwarekomponenten:

1. **Inkompatibilität:** Manchmal sind Softwarekomponenten nicht kompatibel mit den Anforderungen oder der Architektur des neuen Systems. In solchen Fällen kann es schwierig sein, die Komponenten anzupassen oder zu integrieren, was zu zusätzlichem Aufwand führen kann.
2. **Mangelnde Flexibilität:** Wenn die vorhandenen Softwarekomponenten nicht ausreichend flexibel sind, um den spezifischen Anforderungen des neuen Systems gerecht zu werden, kann es sinnvoller sein, neue Komponenten zu entwickeln, die besser auf die Bedürfnisse zugeschnitten sind.
3. **Qualitätsprobleme:** Es besteht die Möglichkeit, dass die wiederverwendeten Softwarekomponenten Qualitätsprobleme aufweisen, die sich negativ auf das Gesamtsystem auswirken können. In solchen Fällen kann es besser sein, neue Komponenten zu entwickeln, um sicherzustellen, dass die Qualität und Zuverlässigkeit des Systems gewährleistet sind.
4. **Abhängigkeit von Dritten:** Wenn die wiederverwendeten Softwarekomponenten von Dritten entwickelt wurden, besteht eine gewisse Abhängigkeit von diesen. Änderungen oder Aktualisierungen an den Komponenten können von externen Faktoren abhängen, was die Kontrolle über das System beeinträchtigen kann.

Die Entscheidung, ob Softwarekomponenten wiederverwendet oder neu entwickelt werden sollten, sollte sorgfältig abgewogen werden, unter Berücksichtigung der spezifischen **Anforderungen**, der Kompatibilität, der Flexibilität und der Qualität der vorhandenen Komponenten.

Zu 3.

Ein Benutzungsdiagramm von Softwarekomponenten, auch bekannt als **Komponentendiagramm**, drückt die Beziehungen und Abhängigkeiten zwischen verschiedenen Softwarekomponenten aus. Es zeigt, wie die Komponenten in einem System **zusammenarbeiten** und wie sie miteinander **interagieren**.

Ein Benutzungsdiagramm besteht aus Komponenten, Schnittstellen und deren Beziehungen zueinander. Es zeigt, welche Komponenten von anderen Komponenten abhängig sind und wie sie miteinander kommunizieren. Die Beziehungen können durch Pfeile oder Verbindungen dargestellt werden, die die Richtung der Abhängigkeit oder Kommunikation anzeigen.

Ein Benutzungsdiagramm kann folgende Informationen ausdrücken:

1. **Abhängigkeiten:** Es zeigt, welche Komponenten von anderen Komponenten abhängig sind. Zum Beispiel kann eine Komponente A eine Abhängigkeit von einer anderen Komponente B haben, was bedeutet, dass A B benötigt, um ordnungsgemäß zu funktionieren.
2. **Kommunikation:** Es zeigt, wie Komponenten miteinander kommunizieren. Zum Beispiel kann eine Komponente A eine Schnittstelle von einer anderen Komponente B verwenden, um Daten auszutauschen oder Funktionen aufzurufen.
3. **Wiederverwendung:** Es zeigt, welche Komponenten wiederverwendet werden. Wenn eine Komponente in mehreren Teilen des Systems verwendet wird, kann dies im Benutzungsdiagramm dargestellt werden.

4. **Hierarchie:** Es zeigt die Hierarchie der Komponenten, wenn es eine übergeordnete oder untergeordnete Beziehung zwischen ihnen gibt. Dies kann helfen, die Struktur des Systems zu verstehen.

Ein Benutzungsdiagramm ist ein nützliches Werkzeug, um die Architektur und den Aufbau eines Softwaresystems zu visualisieren. Es hilft Entwicklern und anderen Stakeholdern, die Beziehungen zwischen den Komponenten zu verstehen und die **Auswirkungen von Änderungen oder Erweiterungen auf das System zu analysieren.**

Lösungsansatz zur fallbezogenen Aufgabe

Die Modularität und Wiederverwendung von Softwarekomponenten spielen eine wichtige Rolle bei dem skizzierten **E-Learning-System**. Hier sind einige Aspekte, die darauf hinweisen:

1. **Skalierbarkeit:** Das System umfasst eine große *Anzahl von Benutzern* und verarbeitet eine erhebliche *Menge an Daten*. Durch die Verwendung modularer Komponenten kann das System leichter skaliert werden, um den Anforderungen des *wachsenden Benutzerstamms* gerecht zu werden. Neue Komponenten können hinzugefügt oder vorhandene Komponenten erweitert werden, um die *Leistung und Kapazität des Systems* zu verbessern.
2. **Flexibilität:** Durch die Modularität können einzelne Komponenten *unabhängig* voneinander *entwickelt, getestet und gewartet* werden. Dies ermöglicht eine *schnellere Entwicklung* und Bereitstellung neuer Funktionen oder Dienste. Es erleichtert auch die Anpassung des Systems an sich ändernde Anforderungen oder *Technologien*.
3. **Wiederverwendung:** Das System basiert auf Open-Source-Software und nutzt bereits vorhandene Komponenten, die von anderen Universitäten entwickelt wurden. Dies ermöglicht eine effiziente Wiederverwendung von bewährten Lösungen und vermeidet die Notwendigkeit, alles von Grund auf neu zu entwickeln. Die Wiederverwendung von Softwarekomponenten *spart Zeit und Ressourcen und verbessert die Qualität und Zuverlässigkeit des Systems*.
4. **Erweiterbarkeit:** Durch die modulare Architektur können neue Komponenten entwickelt und in das System integriert werden, um zusätzliche Funktionen oder Dienste bereitzustellen. Dies ermöglicht eine *kontinuierliche Weiterentwicklung* des Systems, um den sich ändernden Anforderungen der Benutzer gerecht zu werden.
5. **Gemeinschaftsentwicklung:** Das System wird gemeinsam mit anderen Universitäten weiterentwickelt. Die Verwendung modularer Komponenten erleichtert die Zusammenarbeit und den Austausch von Erweiterungen und Verbesserungen zwischen den Universitäten. Dies fördert die Zusammenarbeit und den *Wissensaustausch* in der Entwicklergemeinschaft.

Insgesamt ermöglichen die Modularität und Wiederverwendung von Softwarekomponenten eine **effiziente Entwicklung, Skalierung und Anpassung des E-Learning-Systems**. Sie tragen zur Flexibilität, Erweiterbarkeit und Qualität des Systems bei und fördern die *Zusammenarbeit* in der Entwicklergemeinschaft.

Lösungsblatt: Was hat das mit Softwareentwicklung zu tun?

Probleme der Softwaretechnik

- ✚ Komplexität
- ✚ viele Funktionen
- ✚ viele, sich oft widersprechende Zielsetzungen
- ✚ viele, in sich bereits komplexe Komponenten
- ✚ viele verschiedene Personen aus verschiedenen Fachrichtungen
- ✚ oft jahrelanger Entwicklungsprozess und Softwarelebenszyklus

Veränderung

- ✚ Anforderungen ändern sich wenn Fehler entdeckt werden oder die Entwickler sie besser verstehen.
- ✚ Lange Projektdauer -> Oft wechselnde Mitarbeiter
- ✚ schnelle technologische Änderungen
- ✚ Das Einfrieren von Anforderungen resultiert in einem unbrauchbaren System

Beschränkte Ressourcen

- ✚ Geld, Zeit, Personen

Fehler

- ✚ Anwender, Programmierer, Management

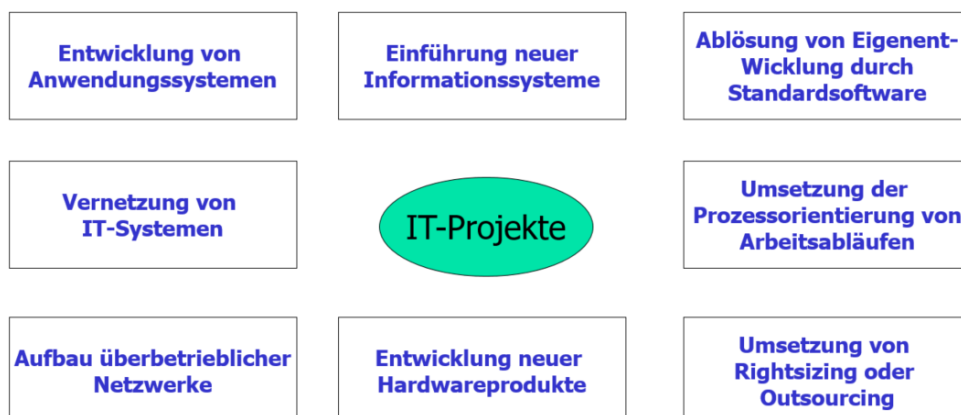
Folgerung:

Wir erlernen Methoden und Verfahren um Softwareprojekte in allen Phasen des Softwareentwicklungsprozesses zu unterstützen und damit die Probleme beherrschbar zu machen.

Bestandteile des Softwareentwicklungsprozesses

- Ablaufplanung von SW-Projekten
- Machbarkeit von SW-Projekten
- Planung von Ressourcen
- Analyse von Anforderungen
- Modellierung/Entwurf von Software
- Dokumentation
- Fehlersuche
- etc.

Beispiele für IT-Projekte



Lösungsblatt: Was ist Software? Um welche Art von Software handelt es sich hier?

Systemsoftware

Systemsoftware ist zum **Betrieb** und zur **Steuerung** der Hardware erforderlich und umfasst **hardwarenahe, anwendungsneutrale** Entwicklungs- und Verwaltungsprogramme.

Anwendungssoftware

Anwendungssoftware umfasst alle Programme die von **Anwendern** zur **Bearbeitung einer fachlich inhaltlichen Aufgabenstellung** benötigt werden.

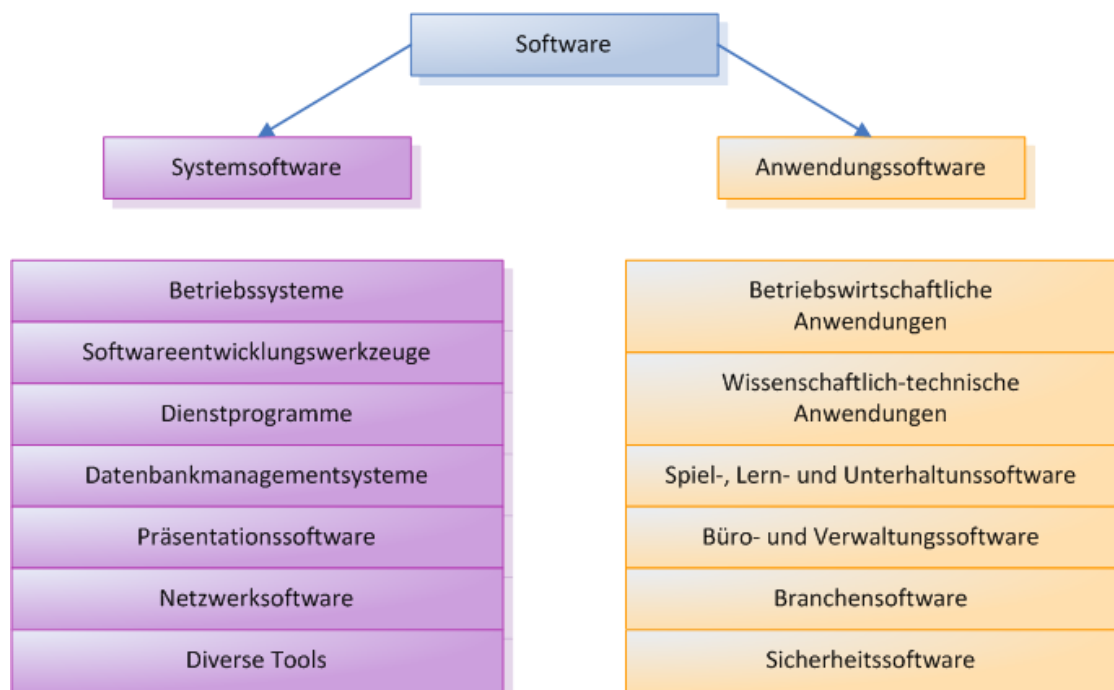


Abbildung 1: Systematik der Software

Systemsoftware wird heute fast ausschließlich als **Standardsoftware** vertrieben, während **Anwendungssoftware** nicht für alle Problemstellungen standardisiert vorhanden ist und dann **individuell entwickelt** oder zumindest stark modifiziert wird.

Standard- oder Individualsoftware

Nach dem **Ursprung** der Anwendungssoftware unterscheidet man Standard- und Individualsoftware.

Individualsoftware

Individualsoftware wird für ein **einzelnes Unternehmen** bzw. für eine **spezialisierte Aufgabenstellung** entwickelt.

Standardsoftware

Unter Standardsoftware werden Softwaresysteme verstanden, die einen **klar definierten Anwendungsbereich** abdecken und als **vorgefertigte** Produkte erworben werden können. Sie werden von vielen Unternehmen für vergleichbare Problemstellungen genutzt.

Software Engineering: Bestandteile von Software

Aufbau und Funktionsweise von Software

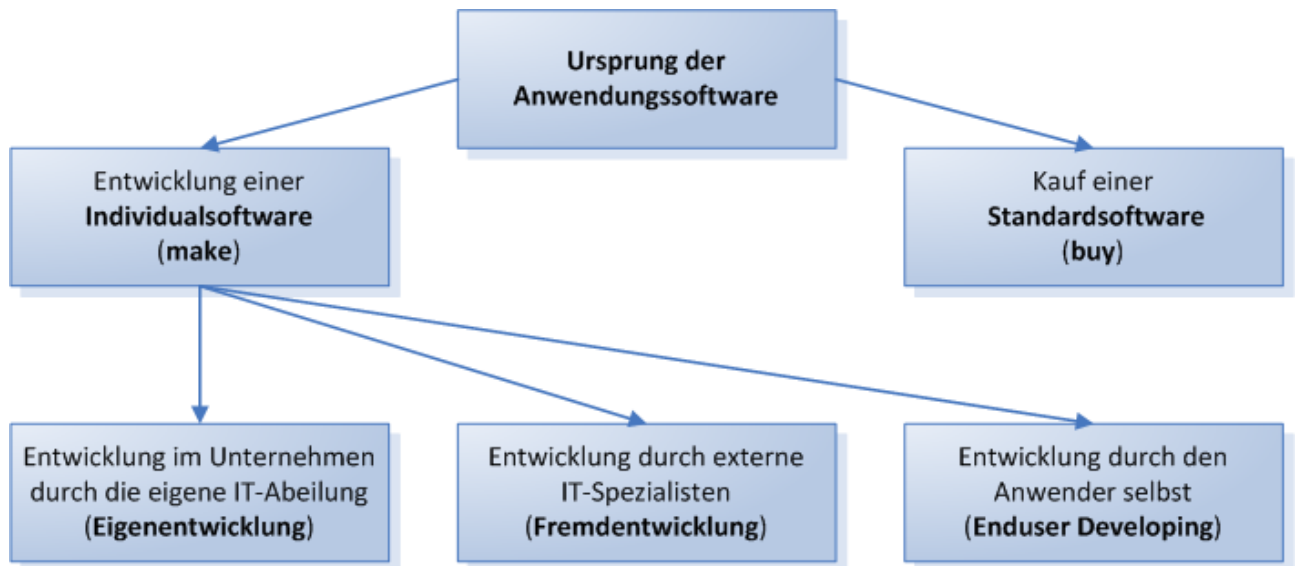


Abbildung 2: Ursprung von Anwendungssoftware

"**Make or buy**"- Entscheidungen, insbesondere bei betriebswirtschaftlicher Software, sind Entscheidungen mit langen Zeithorizont. Daher spielt die Investitionssicherheit eine entscheidende Rolle. Zu bedenken sind dabei die Aufwände für **Lizenz- oder Entwicklungskosten**, Umstellung der internen Organisation auf die Anforderungen der Software sowie die **Schulung der Mitarbeiter**.

Für die Entscheidung zwischen dem Einsatz einer **Standardsoftware** und der **Entwicklung einer Individualsoftware** ist es daher sinnvoll, die jeweiligen **Vor- und Nachteile** aufzulisten und individuell für das Unternehmen zu bewerten.

Nennen Sie jeweils 2 Vor- bzw. Nachteile des Einsatzes einer Standardsoftware	
Vorteile	Nachteile
- relativ geringe Kosten der Anschaffung kein Entwicklungsaufwand im Unternehmen - volle Verfügbarkeit des Funktionsumfangs zum geplanten Einsatzzeitpunkt - Nutzung von Know-how zur Bearbeitung von bestimmten Aufgaben Anpassung und Weiterentwicklung ist im allgemeinen sicher gestellt	- evtl. hohe Kosten der Anpassung - kein Wettbewerbsvorteil - viele ungenutzte Funktionalitäten Abhängigkeit vom Softwareanbieter. Dieser bestimmt den Innovationszyklus und die Nutzbarkeit der Software.

Software Engineering: Bestandteile von Software

Aufbau und Funktionsweise von Software

Individualsoftware	
Vorteile	Nachteile
- Spezieller Zuschnitt auf Unternehmensaufgaben und damit optimaler Ressourceneinsatz. - Wettbewerbsvorteile durch Abbilden von Alleinstellungsmerkmalen Fremdentwicklungen bringen externes Spezialwissen in das Unternehmen - werden effizient unter Verwendung von vorgefertigten Bausteinen (Business Objects) erstellt Eigenentwicklungen keine Preisgabe von internem Know-how - keine Abhängigkeit von unternehmensexternen Leistungen End User Development schnell, flexibel kein Abstimmungsaufwand Unterstützung durch spezielle Tools in Anwendersoftware (z.B. Makros)	hohe einmalige Entwicklungskosten keine/verspätete Verfügbarkeit des Funktionsumfangs zum geplanten Einsatzzeitpunkt Fehleranfälliger, da ein breiter Anwenderkreis als Tester ausfällt - Zur Weiterentwicklung und Wartung ist Personal nötig, das zu erheblichen laufenden Kosten führen kann. - Für Eigenentwicklungen fehlt Zeit oder Kompetenz End User Development nur für kleinere Aufgaben geeignet

Aufgabe zur Branchensoftware

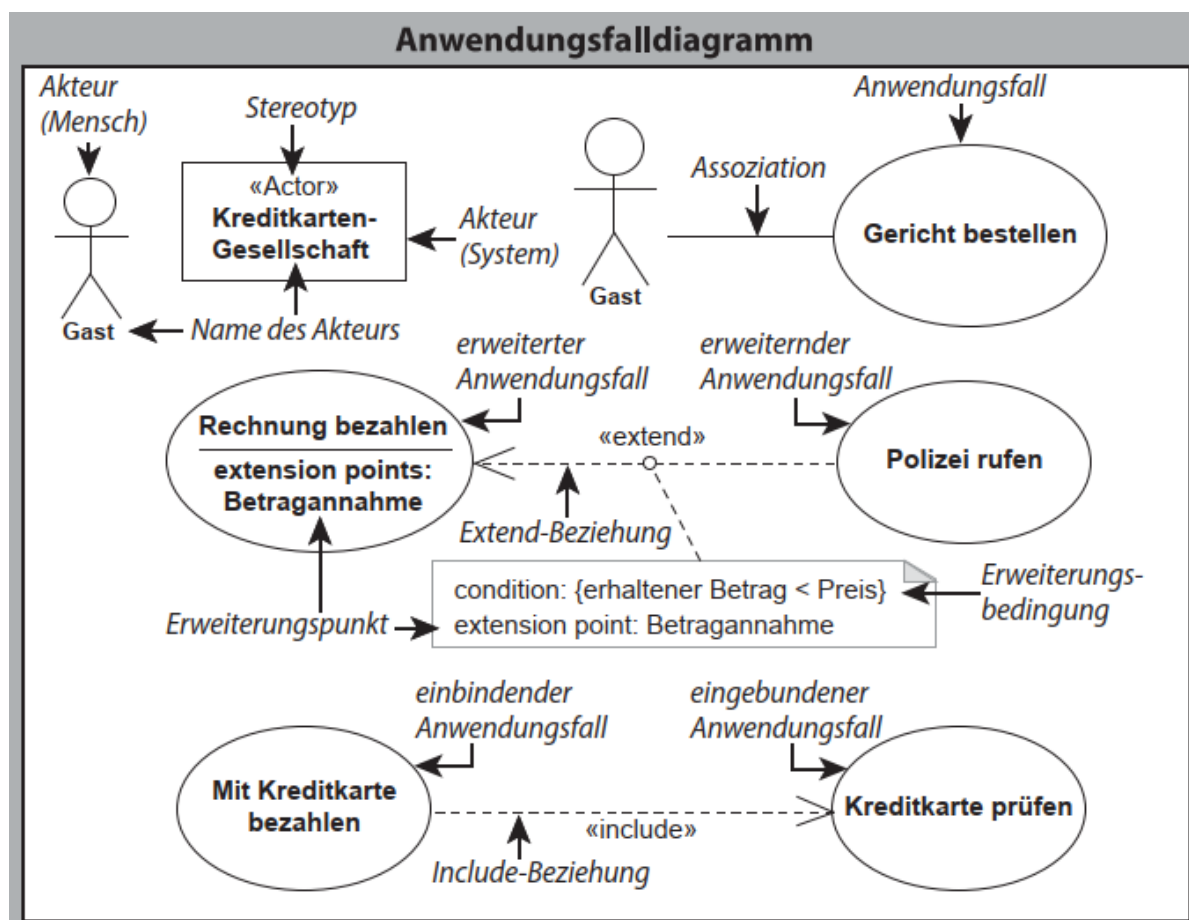
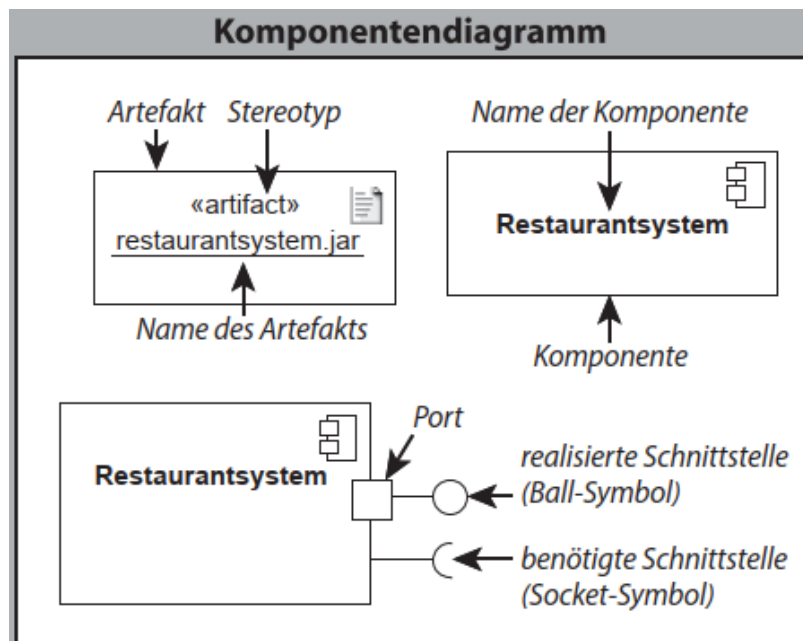
Definieren Sie den Begriff Branchensoftware. Ist Branchensoftware Individual- oder Standardsoftware? Stellen Sie Vor- und Nachteile von Branchensoftware dar.

Branchensoftware deckt einen **klar definierten Anwendungsbereich** eines bestimmten **Marktsegments** ab und kann als **vorgefertigtes** Produkt erworben werden. Branchensoftware fällt damit in die Kategorie **Standardsoftware**.

Allerdings werden auch **Individuallösungen**, die zunächst für einen einzigen Kunden erstellt wurden, anschließend zur **Nachnutzung** angeboten und als **Branchenlösung** vermarktet. In solchen Fällen ist eine Einordnung in die Kategorie **Standardsoftware** nur sehr eingeschränkt zutreffend.

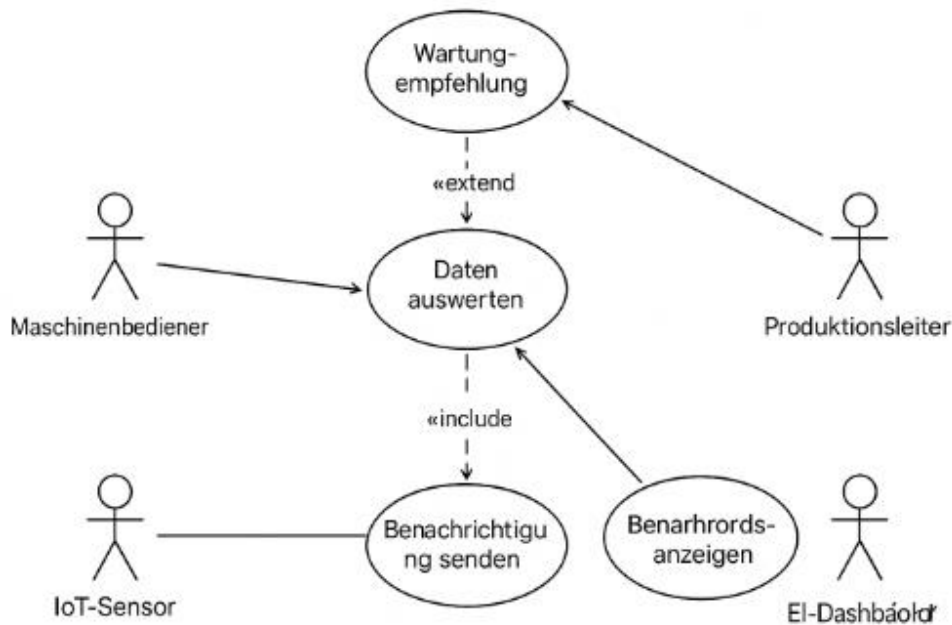
Branchensoftware	
Vorteile	Nachteile
- kostengünstiger als Individualsoftware - gut an die Anforderungen der Branche angepasst - kurzfristig verfügbar - Ein Kunde kauft günstig branchenspezifisches Know-how	- Bindung an technische Plattform des Anbieters - mehrere Branchenlösungen führen zu einer uneinheitlichen IT-Landschaft was die Betriebskosten erhöht und den Zugriff auf Unternehmensinformationen erschwert

Anhang: Benutzungs- und Komponentendiagramm



Beispiele (Modellierung) Sw-Architektur an Predictive Maintenance

Benutzungsdiagramm / Use Case Diagramm



Akteure:

- **Maschinenbediener** → erhält Warnungen & Statusanzeigen
- **Wartungstechniker** → plant & führt Wartungen durch
- **Produktionsleiter** → analysiert Ausfallrisiken, plant Ressourcen
- **IoT-Sensor** (externer Akteur/System) → liefert Zustandsdaten der Maschine

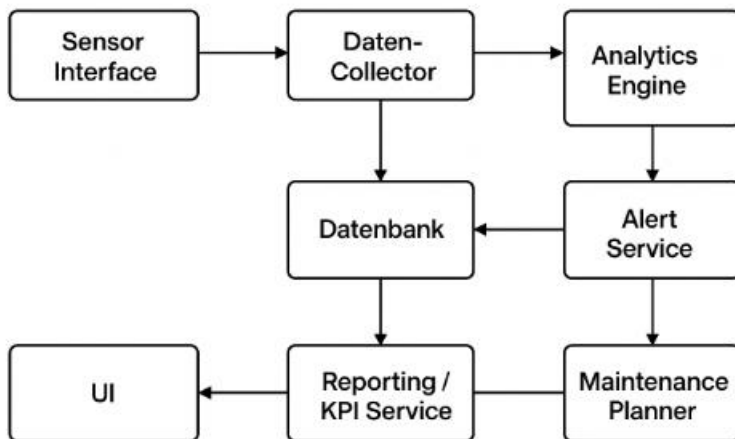
Use Cases:

- **Sensordaten erfassen** (IoT-Sensor → System)
- **Daten auswerten** (System analysiert Verschleiß, Abweichungen)
- **Wartungsempfehlung generieren**
- **Benachrichtigung senden** (an Bediener/Techniker)
- **Dashboards anzeigen** (für Produktionsleiter)

Beziehungen:

- „Daten auswerten“ <<include>> „Sensordaten erfassen“
- „Wartungsempfehlung generieren“ <<extend>> „Daten auswerten“

Komponentendiagramm



Da wir im **monolithischen Szenario** bleiben, bilden wir die **internen** Module **logisch** ab (alles läuft in einer Anwendung, z. B. ein Java- oder .NET-Monolith).

Komponenten:

1. Datenaufnahme-Komponente

- Schnittstelle: Sensor-API
- Aufgabe: Liest Sensordaten (Temperatur, Vibration, Laufzeiten).

2. Datenmanagement & Speicherung

- Schnittstelle: Datenbank-Connector
- Aufgabe: Speicherung historischer Maschinendaten.

3. Analyse-Engine

- Schnittstelle: Predictive Analytics API
- Aufgabe: Mustererkennung, Prognosen, Anomalieerkennung (z. B. ML-Modell).

4. Benachrichtigungs-Komponente

- Schnittstellen: E-Mail, App-Push, Dashboard-API
- Aufgabe: Meldungen an Bediener, Techniker, Manager.

5. UI-Komponente (Dashboard)

- Schnittstelle: Webfrontend
- Aufgabe: Visualisierung von Wartungsstatus, KPIs, Empfehlungen.

Anschlussmöglichkeiten Sensor

